

**UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II**



**SCUOLA POLITECNICA DELLE SCIENZE
DI BASE**

**CORSO DI LAUREA MAGISTRALE IN INGEGNERIA DELL'AUTOMAZIONE E
ROBOTICA**

**DIPARTIMENTO DI INGEGNERIA ELETTRICA
E TECNOLOGIE DELL'INFORMAZIONE**

FIELD & SERVICE ROBOTICS

**CAR-LIKE ROBOT:
PLANNING, CONTROL & LOCALIZATION**

Professor

Prof. Fabio Ruggiero

Candidates

Giuseppe Arena [P38000311]
Nicholas Ruggiero [P38000312]

ACADEMIC YEAR 2025/2026

Contents

1	Kinematic Model	1
2	Planning	3
2.1	Map Processing	3
2.2	Graph Generation via Reeds-Shepp	4
2.3	Optimal Path Search via Dijkstra	5
2.4	Trajectory Generation and Velocity Profiles	7
2.5	Uniform Scaling for Physical Constraints	8
3	Control	10
3.1	Controller Architecture	11
3.1.1	Computation of Desired Outputs	11
3.1.2	Computation of Actual Outputs	11
3.1.3	Input/Output Controller	11
3.1.4	Computation of Physical Velocities	12
3.2	Ideal Feedback Results	13
4	Localization	16
4.1	Odometry	16
4.1.1	Forward Eulero Odometry	16
4.1.2	Eulero Odometry Results	17
4.1.3	Runge-Kutta 2 (RK2) Odometry	20
4.1.4	Runge-Kutta Odometry Results	20
4.2	Active Localization with Extended Kalman Filter	23
4.2.1	Simulated LiDAR Sensor	23
4.2.2	EKF Implementation	24
4.2.3	Active Localization Results	24
5	Conclusion	27
5.1	Evaluation of the I/O Feedback Controller	27
5.2	Comparison of Localization Methods	28

Introduction

The autonomous navigation of non-holonomic vehicles in complex environments requires a robust architecture capable of integrating motion planning, smooth trajectory generation, and stable control systems. This project tackles the problem of designing a complete guidance and control framework for a car-like robot operating in a mapped workspace.

The primary objective is to drive the vehicle from a starting configuration to a designated goal while strictly avoiding obstacles and respecting the mechanical constraints of the robot, such as its minimum turning radius and maximum velocity limits. To achieve this, the proposed methodology is divided into sequential stages. First, a geometric path is synthesized utilizing a graph-search approach based on Reeds-Shepp curves, ensuring that the kinematic limits of the vehicle are inherently satisfied. Subsequently, a safe and smooth time law is assigned to the path using fifth-order polynomials over macro-segments, effectively managing cusps (direction reversals) and preventing sudden acceleration spikes. Finally, a reference trajectory is obtained and dynamically scaled to guarantee physical feasibility.

The overall layout of the proposed architecture, which acts as the foundation for the subsequent tracking and localization modules, provides a reliable and realistic simulation environment for advanced robotic motion control.

Chapter 1

Kinematic Model

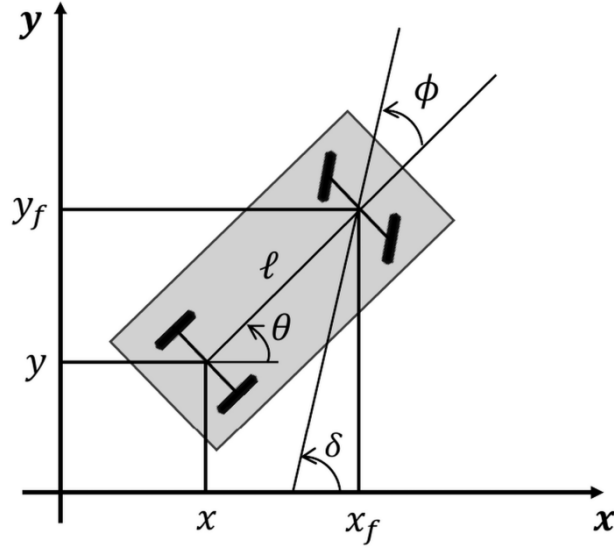


Figure 1.1: Car-like kinematic model.

To accurately control the vehicle, its motion must be modeled mathematically. The car-like robot is traditionally represented using the Ackermann steering geometry, which can be effectively simplified into a non-holonomic "bicycle model" by collapsing the front and rear wheel pairs onto the longitudinal axis.

Let $\mathbf{q} = [x, y, \theta, \phi]^T$ be the generalized state vector, where (x, y) are the Cartesian coordinates of the center of the rear axle, θ represents the orientation (heading) of the vehicle body with respect to the global X -axis, and ϕ is the steering angle of the front virtual wheel.

The system is governed by pure rolling conditions, which imply that the wheels cannot slip laterally. These non-holonomic constraints are expressed in Pfaffian

form as:

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0 \quad (1.1)$$

$$\dot{x}_f \sin(\theta + \phi) - \dot{y}_f \cos(\theta + \phi) = 0 \quad (1.2)$$

where (x_f, y_f) denote the coordinates of the center of the front axle. By defining l as the wheelbase (the distance between the front and rear axles), the forward kinematics mapping the control inputs $\mathbf{u} = [v, \omega]^T$ (linear driving velocity at the rear axle and steering angular velocity) to the state derivatives is formulated as follows:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \\ \frac{\tan \phi}{l} \\ 0 \end{bmatrix} v + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \omega \quad (1.3)$$

This representation captures the essence of the vehicle's kinematic limitations: it cannot move directly sideways (as dictated by the non-holonomic constraint), and its rate of heading change, explicitly defined as $\dot{\theta} = \frac{v}{l} \tan \phi$, is strictly coupled with its forward speed v and the current steering angle ϕ . This rigorous mathematical model serves as the foundational core for both the trajectory tracking feedback linearization and the odometric state estimation (via Euler and RK2 integration) employed in this study.

Chapter 2

Planning

The planning phase is responsible for computing a safe, feasible, and optimized trajectory. This process transforms a raw environmental map into a continuous, time-parameterized reference state vector that the low-level controllers can track.

2.1 Map Processing

The environment is initially provided as a real-world map extraction. To make it processable by the path-finding algorithms, the image is imported and converted into a binary occupancy grid. During this binarization process, roads are assigned a logical value of 1, whereas unnavigable areas and obstacles are assigned a value of 0. Additionally, the resulting matrix is transposed to properly align with the Cartesian (x, y) coordinate system. This binary matrix acts as the absolute collision-checking reference for all subsequent path expansions. Then the user is prompted to manually select the Start and Goal points directly on the map interface, ensuring flexibility across different navigation scenarios.

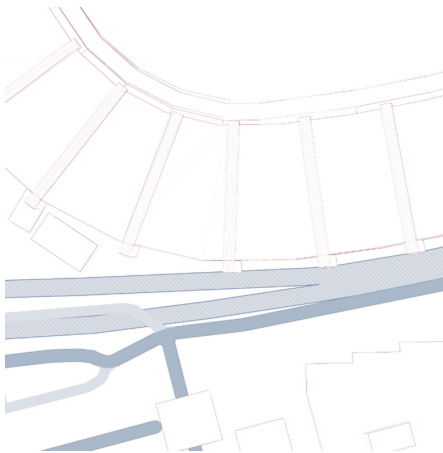


Figure 2.1: Original map

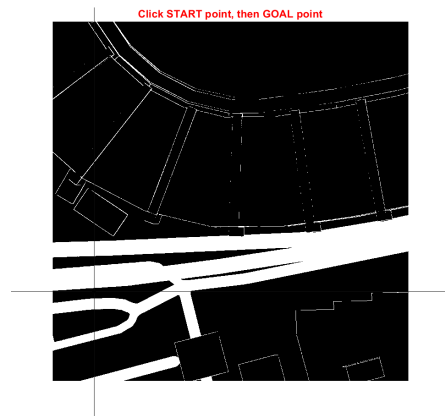


Figure 2.2: Binarized map.

2.2 Graph Generation via Reeds-Shepp

Because the car-like robot is subject to non-holonomic constraints, it cannot move laterally and must strictly respect a minimum turning radius. Traditional grid-based algorithms are therefore inadequate, as they generate sharp corners and cusps that the vehicle cannot physically execute without complex maneuvering. To overcome this, the planning algorithm constructs a customized graph directly tailored to the vehicle’s kinematics.

Instead of randomly sampling points in the workspace, the implemented algorithm builds a reachability graph through an iterative forward-simulation of Reeds-Shepp motion primitives. Starting from the initial pose, the robot’s future states are explored by applying three specific control actions at each iteration: moving straight forward, turning left, or turning right.

The primitives are geometrically constrained by the vehicle’s physical limits: straight steps are set to a fixed length (7 meters), while curves are generated using the minimum turning radius ($R_{min} = 7$ m) and span a fixed angular variation of $\pi/2$ radians (quarter circles).

To guarantee obstacle avoidance, every generated primitive is densely discretized into intermediate waypoints (evaluating 300 points for straight segments and 1001 points for curves). Each of these points is cross-referenced with the binarized occupancy grid. If any point of the curve falls onto a cell marked as an obstacle, that specific branch is immediately discarded. If the primitive is collision-free and its endpoint has not been previously visited, the new node is added to an exploration queue. This iterative breadth-first expansion continues up to a maximum of 500 iterations, progressively building a tree of kinematically executable and safe routes that densely cover the viable space.

2.3 Optimal Path Search via Dijkstra

Once the graph of feasible Reeds-Shepp primitives is fully generated, the spatial relationships between the valid nodes are formalized into an adjacency matrix. In this specific implementation, the connections are unweighted (assigned a uniform cost of 1), meaning the subsequent search will naturally optimize for the path requiring the fewest number of discrete maneuvers, rather than strictly minimizing the Euclidean distance.

Due to the discrete nature of the generated motion tree, it is highly unlikely that a node will land exactly on the user-defined goal coordinates. To address this, the algorithm calculates the Euclidean distance from all generated nodes to the target. The node closest to the target is selected as the designated arrival point, provided it falls within a maximum acceptable safety tolerance (25 meters). If no node is generated within this radius, the path is considered unfeasible.

With the start and designated goal nodes defined, Dijkstra’s algorithm is applied to the adjacency matrix to find the optimal sequence of primitives. The algorithm initializes all node distances to infinity, except for the start node, and utilizes a queue of unvisited nodes. At each step, it extracts the node with the lowest cumulative cost and evaluates its connected neighbors. If a neighbor can be reached with a lower cost through the current node, its shortest distance is updated, and the current node is recorded as its predecessor.

This evaluation continues until the goal node is reached or the queue is empty. Finally, the optimal path is reconstructed by backtracking the predecessor references from the goal back to the start, yielding a continuous, collision-free geometric route ready for time-parameterization.

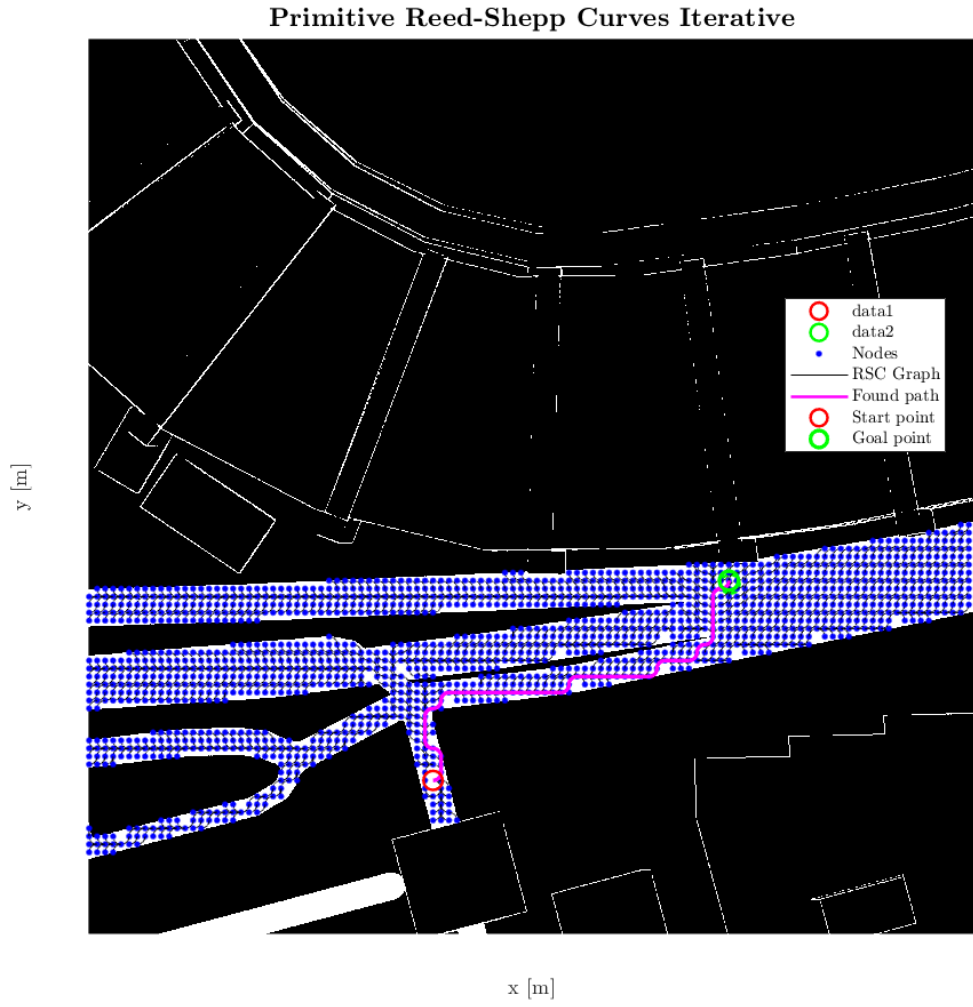


Figure 2.3: Generated Reeds-Shepp graph with the optimal geometric path

2.4 Trajectory Generation and Velocity Profiles

The geometric path returned by Dijkstra consists of a sequence of spatial coordinates. To allow a feedback controller to track this path, it must be parameterized in time, defining exactly when the robot should be at a specific coordinate. Instead of relying on standard polynomial segmentations, a dynamic curvature-based velocity profiling strategy is employed to ensure fluidity and avoid mechanical stress.

First, the path undergoes a spatial parameterization via arc-length calculation. The geometric curvature of the road, denoted as κ , is evaluated numerically using the first and second spatial derivatives:

$$\kappa = \frac{|x'y'' - y'x''|}{(x'^2 + y'^2)^{1.5}} \quad (2.1)$$

Based on this curvature, a target velocity profile is systematically generated. A nominal maximum velocity (e.g., 2.5 m/s) is assigned to straight segments, while a reduced safety velocity (e.g., 0.6 m/s) is strictly enforced in high-curvature regions ($\kappa > 0.04$).

To guarantee continuous acceleration and deceleration ramps, preventing sudden jerks or spikes, a Gaussian moving average filter (spanning 6% of the total path points) is applied to the velocity profile. Since the Gaussian filter alone does not inherently force the boundaries to zero, a raised-cosine window is superimposed at the extremities of the trajectory. This specifically forces the reference velocity to start and end at exactly zero.

Subsequently, the spatial profile is mathematically transformed into the time domain via numerical integration ($dt = ds/v$). Finally, the entire trajectory is obtained, resulting in smooth reference commands for the linear and angular velocities.

During this phase, the algorithm also systematically extracts a set of 8 structural landmarks, placed at a 15-meter orthogonal offset from the generated trajectory. This approach is chosen in order to have the only necessary landmarks for the trajectory chosen by the user, alternatively the landmarks can be set as fixed in the map. These landmarks act as essential detection points for the Active Localization utilized in the final control stages.

2.5 Uniform Scaling for Physical Constraints

The dynamically generated trajectory might mathematically require speeds that exceed the robot's physical capabilities. To prevent actuator saturation, a Uniform Scaling step is enforced.

The maximum required linear velocity (v_{peak}) and angular velocity (ω_{peak}) of the entire trajectory are extracted and compared against the hardware limits ($v_{max} = 3.0$ m/s, $\omega_{max} = 2.0$ rad/s). A scaling factor k is computed as:

$$k = \max \left(1, \frac{v_{peak}}{v_{max}}, \frac{\omega_{peak}}{\omega_{max}} \right) \quad (2.2)$$

If $k > 1$, the total execution time is uniformly stretched by k ($T_{new} = k \cdot T$), while the velocities and accelerations are scaled down accordingly. This ensures the geometric path remains perfectly unaltered while strictly guaranteeing kinematic safety.

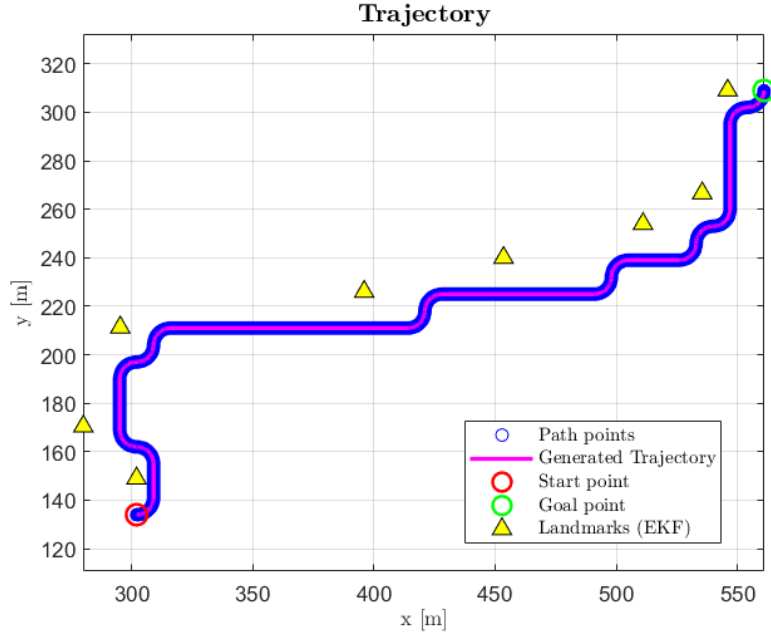


Figure 2.4: Reference trajectory computed

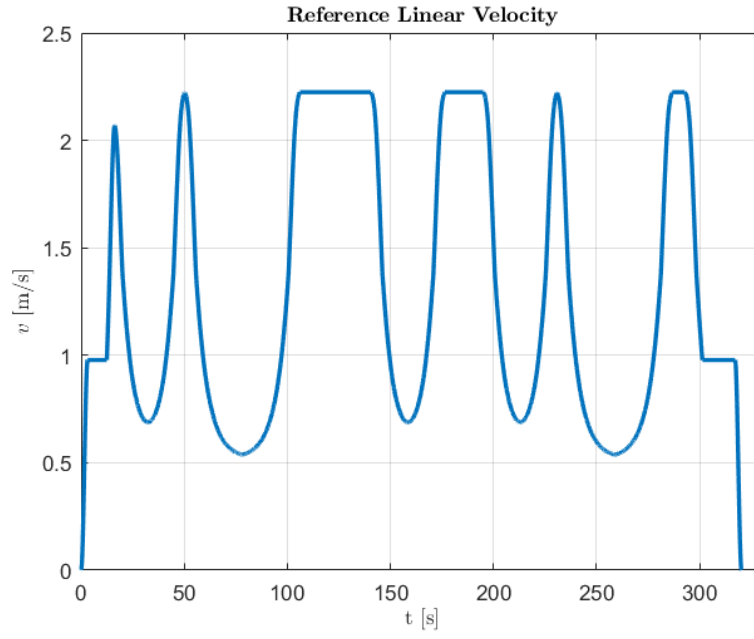


Figure 2.5: Reference linear velocity $v(t)$.

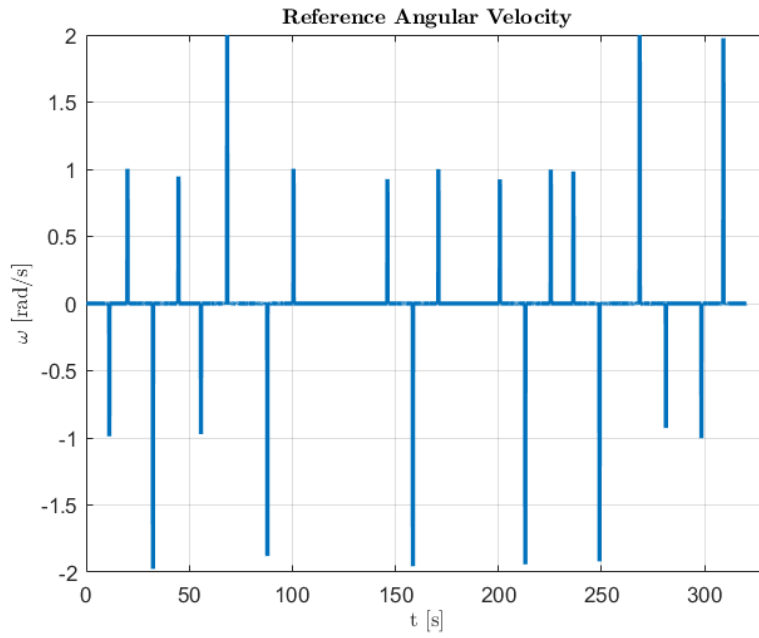


Figure 2.6: Reference steering angular velocity $\omega(t)$.

Chapter 3

Control

To enable the car-like robot to accurately track the generated reference trajectory, an Input-Output (I/O) Feedback Linearization controller is implemented. The kinematic model of a car-like vehicle is non-holonomic, meaning it cannot be stabilized to a point or follow an arbitrary path using smooth, time-invariant feedback directly on its center of mass or rear axle. This limitation arises because the decoupling matrix becomes singular.

To overcome this structural singularity, the I/O Linearization technique shifts the controlled output to a virtual point located ahead of the vehicle. By doing so, the system is mathematically transformed into a linear, decoupled system, allowing the application of standard linear control techniques.

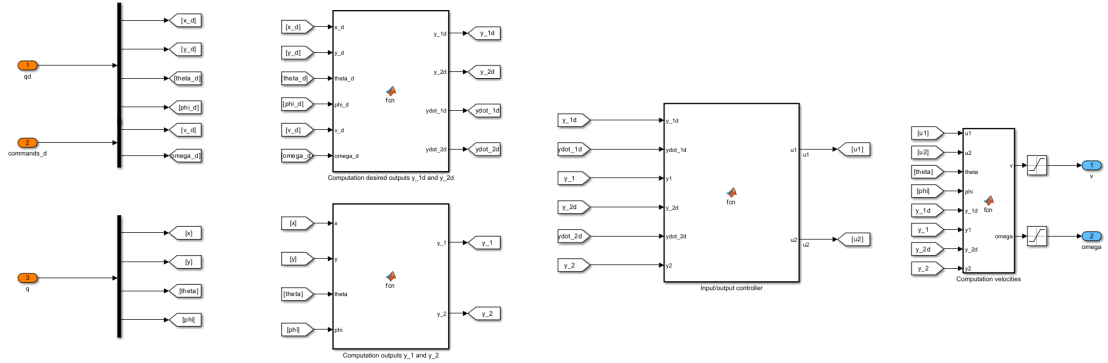


Figure 3.1: IO Feedback Linearization Controller: Simulink Scheme.

3.1 Controller Architecture

The controller is divided into four main functional blocks, each responsible for a specific step in the linearization and tracking process.

3.1.1 Computation of Desired Outputs

The first step is to translate the reference state vector $(x_d, y_d, \theta_d, \phi_d)$ into the desired trajectory for the virtual control point, denoted as (y_{1d}, y_{2d}) . This point is defined by projecting forward from the rear axle by the wheelbase distance l , and then further extending by a look-ahead distance Δ along the orientation of the steering wheels:

$$y_{1d} = x_d + l \cos(\theta_d) + \Delta \cos(\theta_d + \phi_d) \quad (3.1)$$

$$y_{2d} = y_d + l \sin(\theta_d) + \Delta \sin(\theta_d + \phi_d) \quad (3.2)$$

To provide a precise feedforward action without numerical spikes, the desired velocity of this virtual point $(\dot{y}_{1d}, \dot{y}_{2d})$ is calculated analytically rather than using numerical differentiation. This is achieved by multiplying the reference physical velocities (v_d, ω_d) by the exact kinematic transformation matrix $T(\theta_d, \phi_d)$:

$$\begin{bmatrix} \dot{y}_{1d} \\ \dot{y}_{2d} \end{bmatrix} = T(\theta_d, \phi_d) \begin{bmatrix} v_d \\ \omega_d \end{bmatrix} \quad (3.3)$$

3.1.2 Computation of Actual Outputs

In parallel, the controller continuously evaluates the actual position of the virtual control point (y_1, y_2) based on the current physical state of the robot (x, y, θ, ϕ) , which is fed back from the sensors or odometry block:

$$y_1 = x + l \cos(\theta) + \Delta \cos(\theta + \phi) \quad (3.4)$$

$$y_2 = y + l \sin(\theta) + \Delta \sin(\theta + \phi) \quad (3.5)$$

3.1.3 Input/Output Controller

With the desired and actual coordinates of the virtual point established, a proportional tracking law is applied. Because the I/O Linearization has effectively decoupled the dynamics at the virtual point, independent proportional controllers with gains $k_1 = 15$ and $k_2 = 15$ are used to compute the auxiliary control inputs (u_1, u_2) :

$$u_1 = \dot{y}_{1d} + k_1(y_{1d} - y_1) \quad (3.6)$$

$$u_2 = \dot{y}_{2d} + k_2(y_{2d} - y_2) \quad (3.7)$$

These auxiliary inputs represent the required Cartesian velocities of the virtual point to aggressively eliminate the tracking error.

3.1.4 Computation of Physical Velocities

The final step is to map the computed auxiliary inputs (u_1, u_2) back into the physical control variables of the robot: the linear driving velocity v and the steering angular velocity ω . This is accomplished by inverting the kinematic transformation matrix:

$$\begin{bmatrix} v_{raw} \\ \omega_{raw} \end{bmatrix} = T^{-1}(\theta, \phi) \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad (3.8)$$

Additionally, a sophisticated target-damping logic is integrated into this block. When the robot reaches the very end of the trajectory, the physical velocities theoretically drop to zero. However, near the goal point, minor measurement noise or residual state errors could be amplified by the inverse matrix T^{-1} , resulting in high-frequency chattering or aggressive velocity spikes. To prevent this, a two-stage damping strategy is implemented.

The first stage monitors the Euclidean distance between the current reference virtual point (y_{1d}, y_{2d}) and its final value at the goal $(y_{1d}^{goal}, y_{2d}^{goal})$. This distance measures how close the reference itself is to the end of the planned trajectory, and is therefore independent of any sensor noise or localization drift. A continuous raised-cosine factor α_{near} transitions smoothly from 0 to 1 as this distance decreases from a high threshold $d_{hi} = 0.15$ m to a low threshold $d_{lo} = 0.05$ m:

$$\alpha_{near} = \begin{cases} 0 & \text{if } d_{ref} \geq d_{hi} \\ \frac{1}{2} \left(1 + \cos \left(\pi \frac{d_{ref} - d_{lo}}{d_{hi} - d_{lo}} \right) \right) & \text{if } d_{lo} < d_{ref} < d_{hi} \\ 1 & \text{if } d_{ref} \leq d_{lo} \end{cases} \quad (3.9)$$

When $\alpha_{near} = 0$ (reference still travelling), no damping is applied and the controller behaves normally. When $\alpha_{near} = 1$ (reference has arrived at the goal), the second stage is activated.

The second stage evaluates the residual tracking error, the Euclidean distance between the current reference virtual point and the actual virtual point (y_1, y_2) . At this point in the execution, any remaining error is attributable solely to measurement noise rather than a legitimate tracking transient. A second continuous raised-cosine factor α_{err} transitions from 1 to 0 as this error decreases

from $e_{hi} = 0.10$ m to $e_{lo} = 0.02$ m:

$$\alpha_{err} = \begin{cases} 1 & \text{if } e \geq e_{hi} \\ \frac{1}{2} \left(1 - \cos \left(\pi \frac{e - e_{lo}}{e_{hi} - e_{lo}} \right) \right) & \text{if } e_{lo} < e < e_{hi} \\ 0 & \text{if } e \leq e_{lo} \end{cases} \quad (3.10)$$

The two stages are combined into a single multiplicative damping factor:

$$\alpha = 1 - \alpha_{near} (1 - \alpha_{err}) \quad (3.11)$$

and the final physical velocities are obtained as:

$$v = \alpha v_{raw}, \quad \omega = \alpha \omega_{raw} \quad (3.12)$$

This formulation guarantees that damping is *never* applied while the robot is still legitimately tracking the trajectory ($\alpha_{near} = 0 \Rightarrow \alpha = 1$). It is activated only once the reference has completed its motion, and even then it scales down the commands progressively as the residual error vanishes, rather than reacting abruptly to instantaneous noise. The result is a smooth, continuous deceleration to rest that prevents actuator chattering without interfering with the tracking performance during the maneuver.

3.2 Ideal Feedback Results

To validate the theoretical correctness and tuning of the I/O Feedback Linearization controller, an initial simulation was conducted under nominal conditions. In this scenario, the controller relies on ideal feedback, meaning it receives the exact true state of the robot without any sensor noise, localization drift, or odometric errors.

As shown in Figure 3.2, the actual trajectory executed by the robot perfectly overlays the desired reference path. The robot smoothly navigates through the map, successfully executing sharp cusps and straight segments without violating the non-holonomic constraints.

The overall tracking performance is quantified in Figure 3.3, which displays the Euclidean distance error over time. The error is strictly bounded and extremely small throughout the entire execution, peaking momentarily during severe direction reversals (cusps) but quickly converging back toward zero thanks to the aggressive proportional gains.

The Cartesian errors e_x and e_y remain firmly bounded within ± 0.02 meters during transients, demonstrating excellent positional adherence. Similarly, the

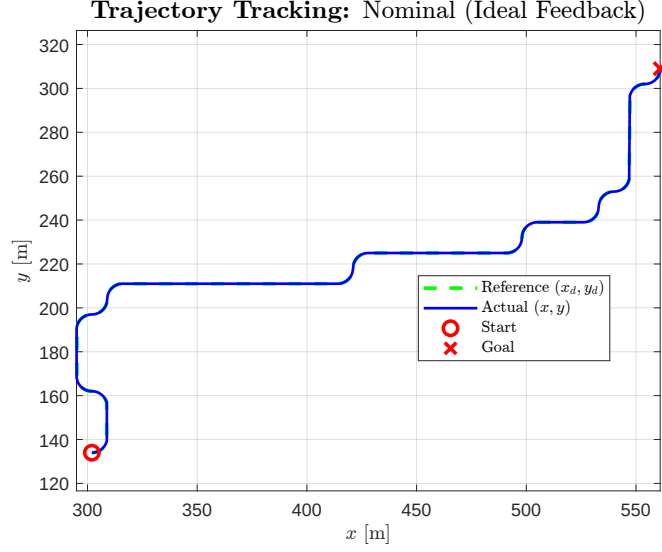


Figure 3.2: Trajectory Tracking: Nominal Case

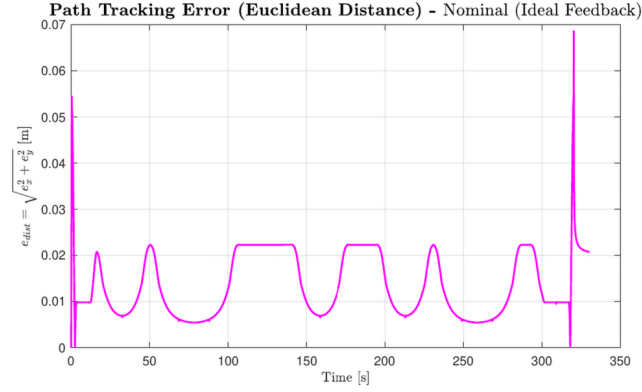


Figure 3.3: Euclidean tracking error over time: Nominal Case

orientation error e_θ and the steering angle error e_ϕ are minimal, showing slight bounded oscillations exclusively during the execution of curved primitives and remaining near zero during straight maneuvers.

The commanded linear velocity v and the steering angular velocity ω are smooth, continuous, and safely contained within the physical limits of the actuators ($v_{max} = 3.0$ m/s, $\omega_{max} = 2.0$ rad/s). The control effort strictly respects the bell-shaped curves mapped during the trajectory generation phase, proving that the analytical feedforward derivatives and the goal-damping logic successfully prevent aggressive spikes or instability.

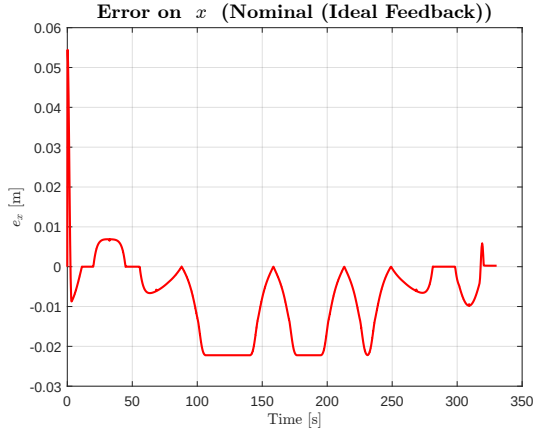


Figure 3.4: x error: Nominal case.

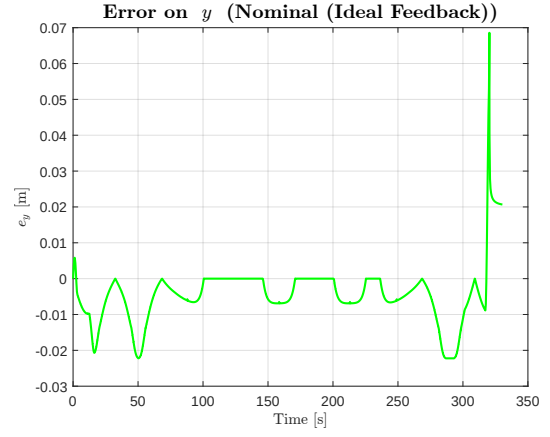


Figure 3.5: y error: Nominal case.

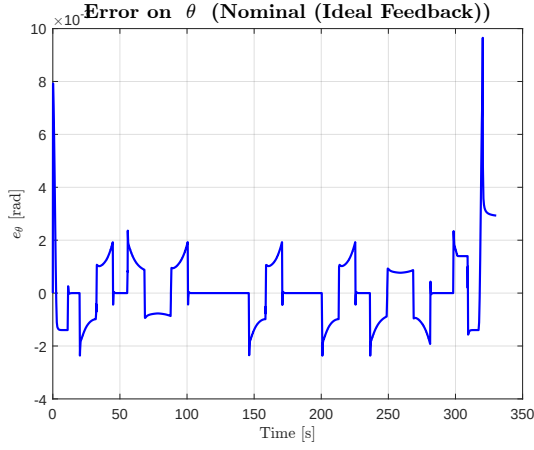


Figure 3.6: θ error: Nominal case.

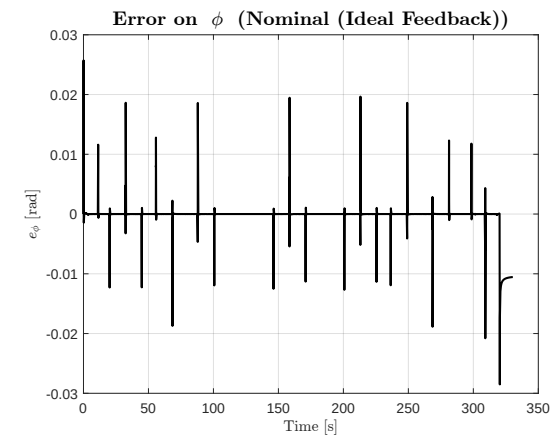


Figure 3.7: ϕ error: Nominal case.

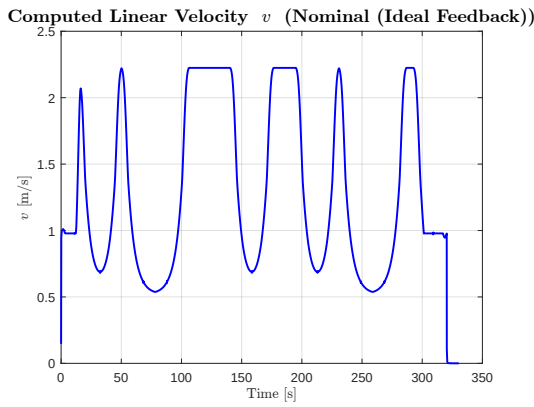


Figure 3.8: $v(t)$ command: Nominal Case.

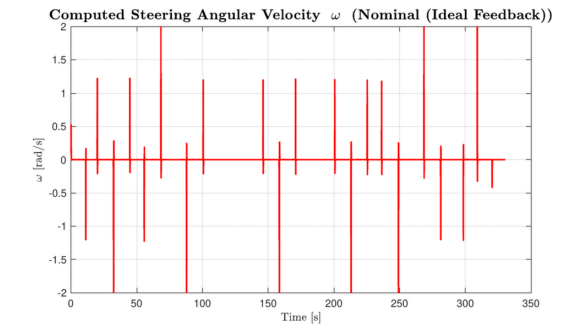


Figure 3.9: $\omega(t)$ command: Nominal Case.

Chapter 4

Localization

In real-world applications, the true state of the robot is not directly accessible. Instead, the vehicle must estimate its current pose and configuration (x, y, θ, ϕ) using onboard sensory data. This process, known as localization, is essential for providing the feedback controller with the necessary information to compute the corrective tracking actions.

The most fundamental approach to localization is odometry, that relies on integrating the measured control velocities—specifically the linear velocity v_{mis} and the steering angular velocity ω_{mis} —over time to estimate the robot’s displacement from a known starting position. While computationally efficient, odometry is an open-loop estimation process, making it inherently susceptible to unbounded numerical drift and integration errors over time.

4.1 Odometry

To implement the odometric estimation, the continuous-time kinematic model of the car-like robot must be discretized. The accuracy of the state estimation heavily depends on the chosen numerical integration method. In this project, two different approaches were implemented and compared: the Forward Euler method and the Runge-Kutta 2 (RK2) method.

4.1.1 Forward Euler Odometry

The Forward Euler method is the simplest first-order numerical integration technique. It computes the state at the next discrete time step $k + 1$ by evaluating the kinematic derivatives strictly at the current time step k , multiplied by the sampling time T_s .

Given the previously estimated state vector $\hat{q}_k = [x_k, y_k, \theta_k, \phi_k]^T$ and the

wheelbase l , the discrete update equations are implemented as follows:

$$x_{k+1} = x_k + v_{mis} T_s \cos(\theta_k) \quad (4.1)$$

$$y_{k+1} = y_k + v_{mis} T_s \sin(\theta_k) \quad (4.2)$$

$$\theta_{k+1} = \theta_k + \frac{v_{mis}}{l} \tan(\phi_k) T_s \quad (4.3)$$

$$\phi_{k+1} = \phi_k + \omega_{mis} T_s \quad (4.4)$$

Because it assumes the rates of change remain completely constant throughout the entire interval T_s , the Euler method introduces a truncation error at every step. In highly non-linear trajectories with frequent curves, these errors accumulate rapidly.

4.1.2 Eulero Odometry Results

To evaluate the performance of the controller under dead-reckoning, a simulation was conducted using the Euler odometry method. In this scenario, the controller relies on the integrated estimated state, meaning it receives a state vector affected by numerical drift rather than the exact true state of the robot.

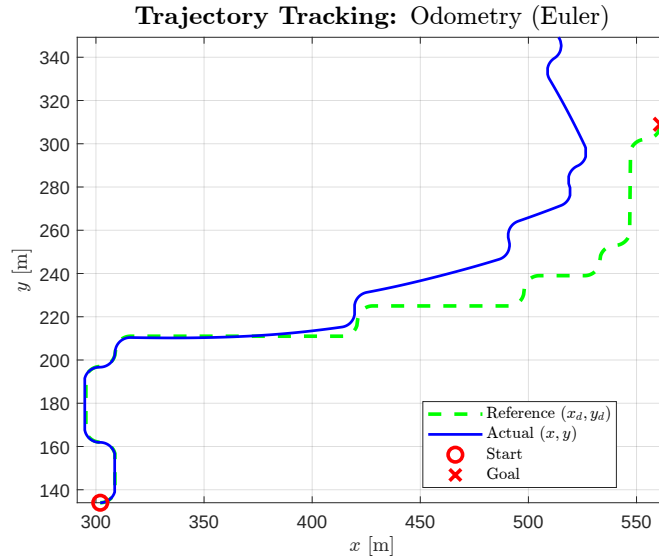


Figure 4.1: Trajectory Tracking: Euler Odometry case

As shown in Figure 4.1, the actual trajectory executed by the robot significantly diverges from the desired reference path. The robot fails to accurately navigate through the map, as the accumulation of numerical errors causes the vehicle to drastically deviate from the correct geometric route.

The overall tracking performance is quantified in Figure 4.2, which displays the Euclidean distance error over time. The error is completely unbounded and grows

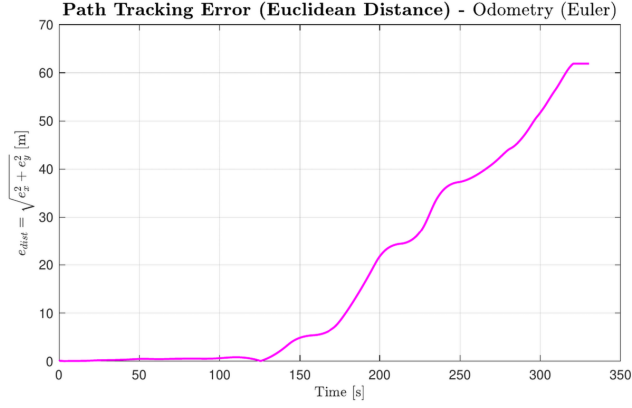


Figure 4.2: Euclidean tracking error: Euler Odometry case.

severely throughout the execution, reaching peaks of over 60 meters, proving that the controller cannot compensate for the continuous open-loop drift generated by the Euler integration.

The Cartesian errors e_x and e_y heavily drift over time, reaching values up to 45 meters and -40 meters respectively, demonstrating a complete loss of positional adherence. Similarly, the orientation error e_θ accumulates significant drift (dropping to -0.6 rad), while the steering angle error e_ϕ presents severe spikes bounded around ± 0.03 rad.

The commanded linear velocity v and the steering angular velocity ω remain contained within the physical limits of the actuators ($v_{max} = 3.0$ m/s, $\omega_{max} = 2.0$ rad/s). The control effort still attempts to follow the generated bell-shaped curves, but since it relies on a completely deteriorated state estimate, the resulting physical movement drives the robot entirely off track.

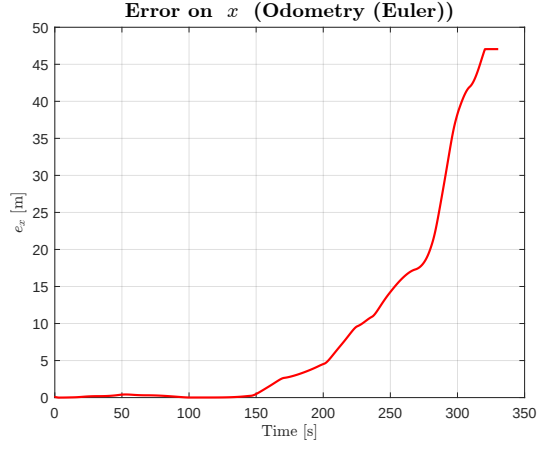


Figure 4.3: x error: Euler case.

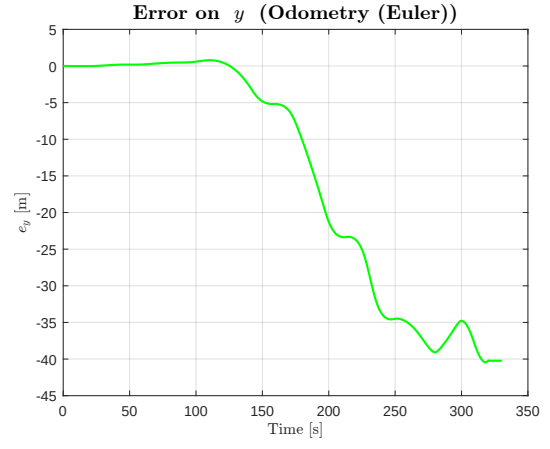


Figure 4.4: y error: Euler case.

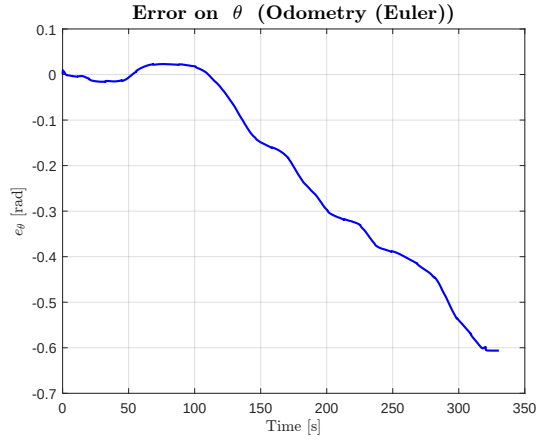


Figure 4.5: θ error: Euler case.

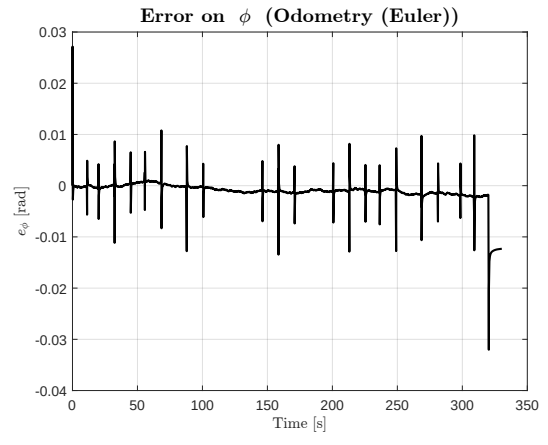


Figure 4.6: ϕ error: Euler case.

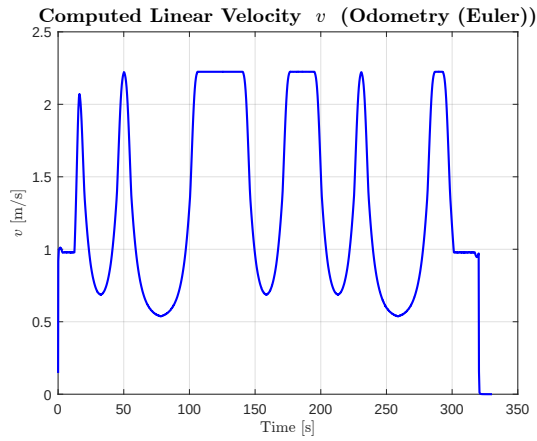


Figure 4.7: $v(t)$ command: Euler case.

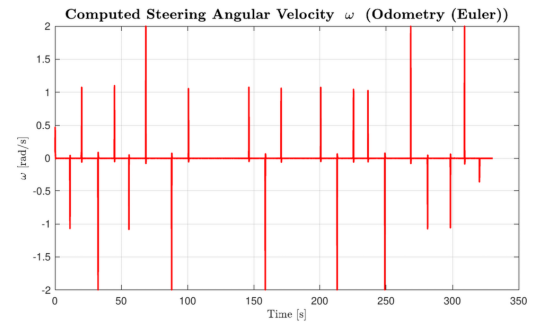


Figure 4.8: $\omega(t)$ command: Euler case.

4.1.3 Runge-Kutta 2 (RK2) Odometry

To improve the numerical accuracy of the state estimation, a second-order Runge-Kutta (RK2) integration method was implemented. Unlike the Forward Euler method, which assumes constant derivatives over the entire time step, the RK2 method evaluates the kinematic rates at the midpoint of the interval to provide a more accurate update.

The RK2 estimation algorithm is divided into two distinct steps. First, the algorithm predicts the intermediate orientation and steering angles at the midpoint of the sampling interval T_s :

$$\phi_m = \phi_k + 0.5 \cdot \omega_{mis} T_s \quad (4.5)$$

$$\theta_m = \theta_k + 0.5 \cdot \frac{v_{mis}}{l} \tan(\phi_k) T_s \quad (4.6)$$

In the second step, these midpoint angles are utilized to compute the final state integration for the next discrete time step $k + 1$. This yields a much better approximation of the true non-linear trajectory of the vehicle:

$$x_{k+1} = x_k + v_{mis} T_s \cos(\theta_m) \quad (4.7)$$

$$y_{k+1} = y_k + v_{mis} T_s \sin(\theta_m) \quad (4.8)$$

$$\theta_{k+1} = \theta_k + \frac{v_{mis}}{l} \tan(\phi_m) T_s \quad (4.9)$$

$$\phi_{k+1} = \phi_k + \omega_{mis} T_s \quad (4.10)$$

By utilizing the midpoint approximation, the RK2 method significantly reduces the local truncation error compared to the first-order Euler approach. However, it remains an open-loop dead-reckoning technique, meaning it cannot correct for accumulated errors over time.

4.1.4 Runge-Kutta Odometry Results

To evaluate the impact of the improved numerical integration on the overall control performance, a simulation was conducted utilizing the RK2 odometry method for state feedback.

As shown in Figure 4.9, despite the higher-order numerical integration, the actual trajectory executed by the robot still diverges drastically from the desired reference path. While RK2 provides a mathematically superior local estimate, the lack of absolute positional correction means that small integration errors continue to accumulate over the long and complex path, causing the vehicle to drift completely off the intended route.

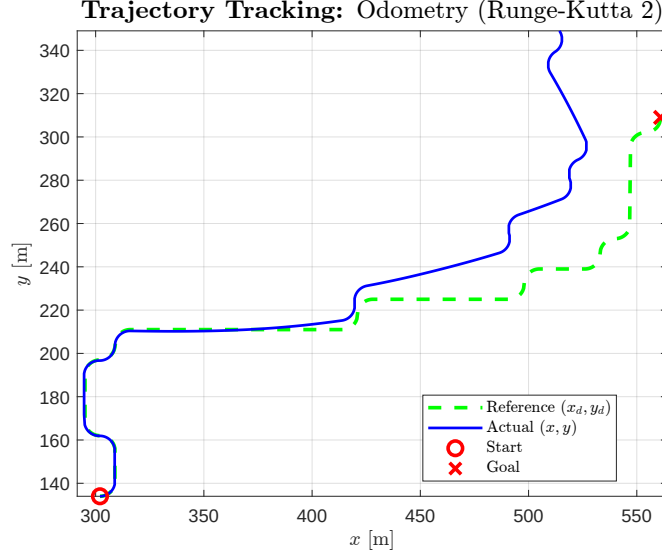


Figure 4.9: Trajectory Tracking: Runge-Kutta 2 Odometry Case

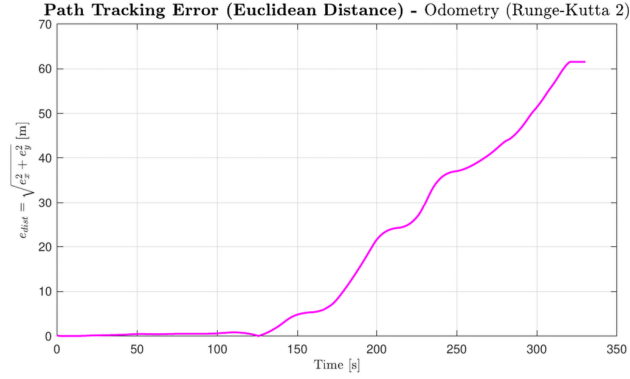


Figure 4.10: Euclidean tracking error: Runge-Kutta 2 Odometry case

Similar to the Euler case, the error is unbounded and grows continuously, reaching a peak deviation of over 60 meters. This clearly demonstrates that improved open-loop integration alone is insufficient for reliable trajectory tracking.

The Cartesian errors e_x and e_y heavily drift over time, reaching extreme values up to approximately 45 meters and -40 meters, respectively, demonstrating a total loss of positional adherence. Similarly, the orientation error e_θ accumulates significant drift (descending to nearly -0.6 rad), while the steering angle error e_ϕ presents sharp spikes bounded around ± 0.03 rad.

The commanded linear velocity v and the steering angular velocity ω are smooth, continuous, and safely contained within the physical limits of the actuators ($v_{max} = 3.0$ m/s, $\omega_{max} = 2.0$ rad/s). However, because the controller is computing these commands based on a heavily degraded state estimate, the resulting physical movement drives the robot entirely off track.

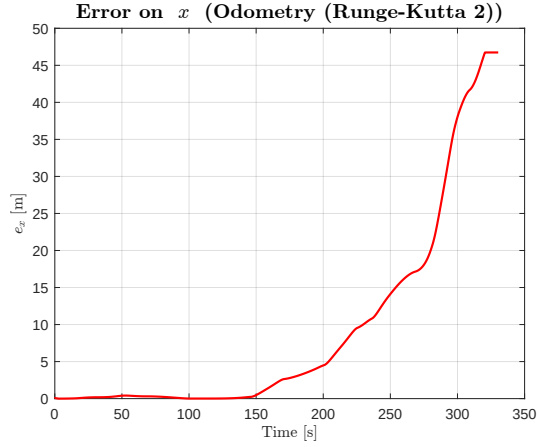


Figure 4.11: x error: Runge-Kutta 2 case.

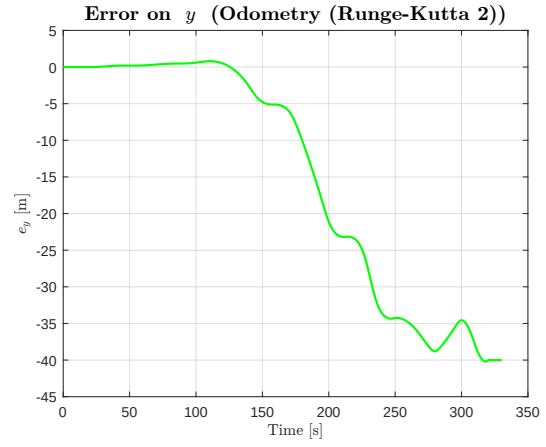


Figure 4.12: y error: Runge-Kutta 2 case.

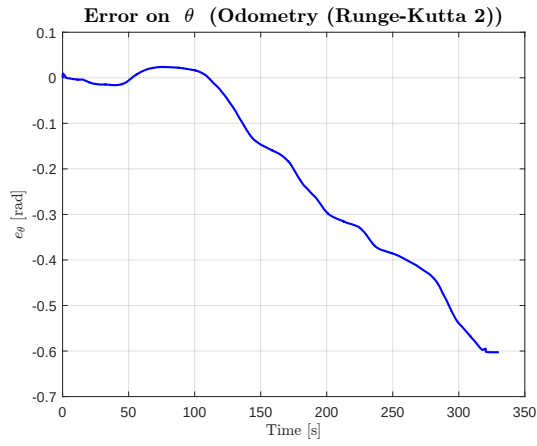


Figure 4.13: θ error: Runge-Kutta 2 case.

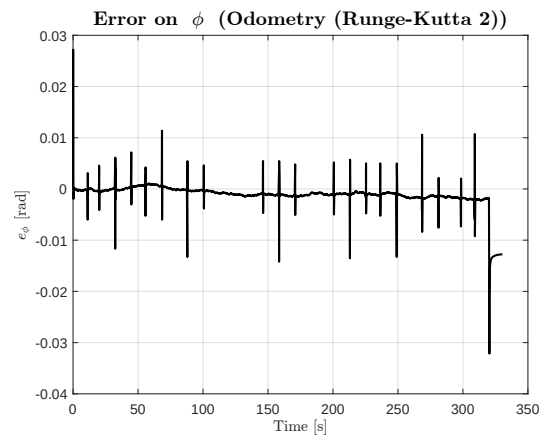


Figure 4.14: ϕ error: Runge-Kutta 2 case.

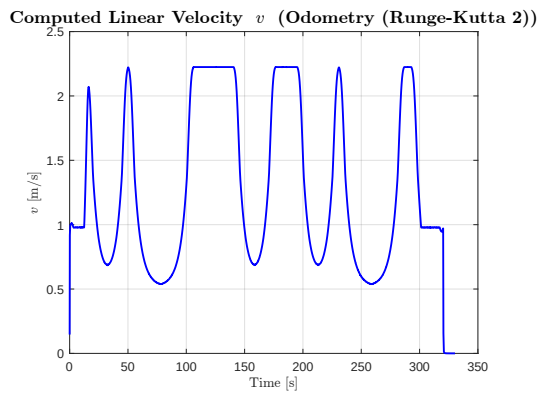


Figure 4.15: $v(t)$ command: Runge-Kutta 2 Case.

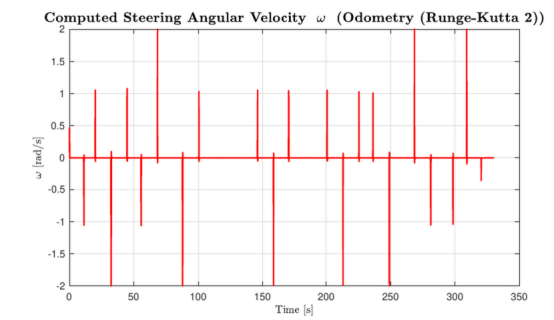


Figure 4.16: $\omega(t)$ command: Runge-Kutta 2 Case.

4.2 Active Localization with Extended Kalman Filter

As demonstrated by the open-loop odometry results, dead-reckoning methods suffer from unbounded error accumulation. To achieve reliable and stable autonomous navigation, the robot must continuously correct its estimated state by observing external references in the environment. This process is known as Active Localization.

To implement this, an Extended Kalman Filter (EKF) is utilized. The EKF is an optimal recursive data processing algorithm capable of fusing proprioceptive information (internal odometry) with exteroceptive information (external sensor measurements). Within the EKF architecture, the Runge-Kutta 2 (RK2) method was specifically chosen for the prediction step. As established in the previous section, RK2 provides a mathematically superior local approximation of the non-linear kinematics compared to the Forward Euler method. By minimizing the local truncation error during the state prediction step, the RK2 method ensures that the filter relies on the best possible dynamic estimate before applying the measurement corrections, thereby improving the overall convergence and stability of the EKF.

4.2.1 Simulated LiDAR Sensor

To provide the EKF with external measurements, a simulated 2D LiDAR sensor was implemented. LiDAR was chosen because it is an industry-standard sensor for autonomous vehicles, capable of highly accurate range and bearing detections.

In the simulation environment, the sensor calculates the relative distance ρ and the relative angle α from the robot's current true position to a set of 8 structural landmarks uniformly distributed along the environment during the planning phase. To replicate real-world sensor imperfections, zero-mean Gaussian noise is injected into the true measurements, scaled by the predefined sensor covariance matrix R_{ekf} .

Additionally, a startup masking logic is incorporated into the sensor acquisition block. For the first few simulation ticks, the measurement noise is suppressed. This strategy prevents the controller from reacting to violent measurement spikes before the EKF covariance matrix has properly initialized and converged, ensuring a smooth departure from the starting position.

4.2.2 EKF Implementation

The Extended Kalman Filter operates in a continuous two-step loop:

1. Prediction Step: The filter first predicts the next state vector $\hat{q}_{pred}$ based on the applied control inputs (v, ω) utilizing the RK2 kinematic integration. Simultaneously, the state covariance matrix P is propagated forward:

$$P_{pred} = F P_{prev} F^T + W Q_{ekf} W^T \quad (4.11)$$

where F is the Jacobian of the motion model with respect to the state, W is the Jacobian with respect to the control inputs, and Q_{ekf} represents the covariance of the process noise (odometric uncertainty).

2. Update Step: When a LiDAR measurement z_{meas} becomes available, the filter sequentially processes each visible landmark. It computes the expected measurement $\hat{z}$ based on the predicted state and evaluates the measurement Jacobian H . The difference between the actual and expected measurement (the innovation) is then weighted by the Kalman Gain K :

$$S = H P_{pred} H^T + R_{ekf} \quad (4.12)$$

$$K = P_{pred} H^T S^{-1} \quad (4.13)$$

Finally, the state estimate and the covariance matrix are formally corrected:

$$\hat{q}_{new} = \hat{q}_{pred} + K(z_{meas} - \hat{z}) \quad (4.14)$$

$$P_{new} = (I - KH)P_{pred} \quad (4.15)$$

4.2.3 Active Localization Results

To validate the theoretical correctness and tuning of the I/O Feedback Linearization controller paired with the EKF state estimator, a final simulation was conducted. In this scenario, the controller relies entirely on the actively localized state feedback, navigating through sensor noise and correcting odometric drift in real-time.

As shown in Figure 4.17, the actual trajectory executed by the robot successfully overlays the desired reference path. Unlike the pure odometry methods, the EKF effectively binds the drift, allowing the robot to accurately reach the goal point while strictly respecting the geometric constraints.

The overall tracking performance is quantified in Figure 4.18, which displays the Euclidean distance error over time. In stark contrast to the unbounded divergence seen in the Euler and RK2 tests, the tracking error under EKF feedback remains strictly bounded throughout the entire execution. The localized peaks

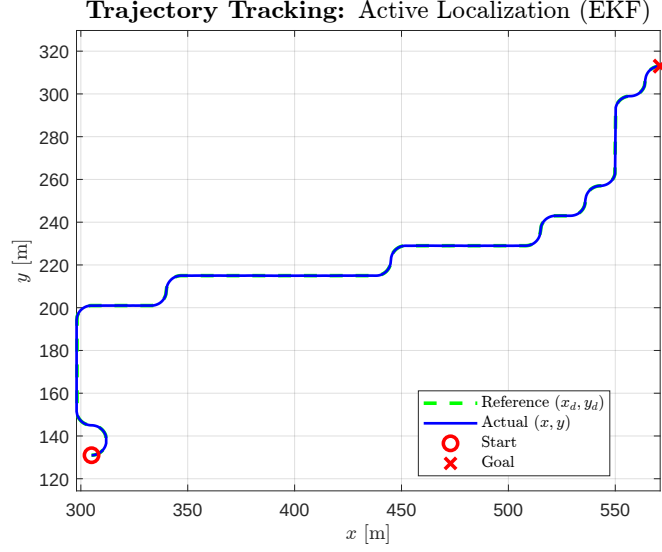


Figure 4.17: Trajectory Tracking: Active Localization case

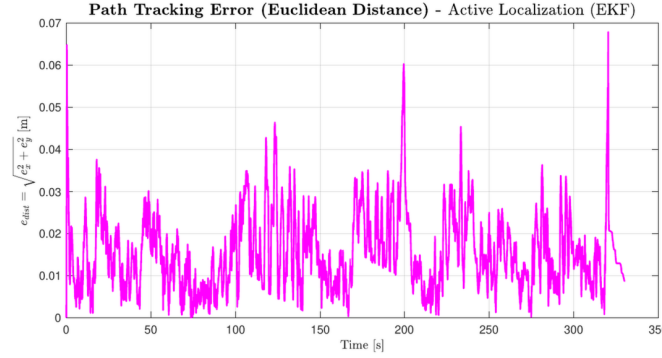


Figure 4.18: Euclidean tracking error: Active Localization case.

represent correction transients during cusps and measurement noise injections, but the filter successfully restrains the overall deviation.

The Cartesian errors e_x and e_y reflect the continuous correction process of the filter. They display high-frequency noise bounded within small margins, demonstrating that the robot is constantly adjusting its position to eliminate the open-loop drift. Similarly, the orientation error e_θ and the steering angle error e_ϕ successfully remain centered around zero, absorbing the measurement uncertainty without destabilizing the non-holonomic progression.

The commanded linear velocity v and the steering angular velocity ω clearly highlight the impact of sensor noise fed back through the aggressive proportional gains of the I/O Linearization controller. While the control signals exhibit noticeable high-frequency chattering due to the stochastic nature of the LiDAR updates, they remain structurally intact and safely contained within the physical saturation limits of the actuators ($v_{max} = 3.0$ m/s, $\omega_{max} = 2.0$ rad/s).

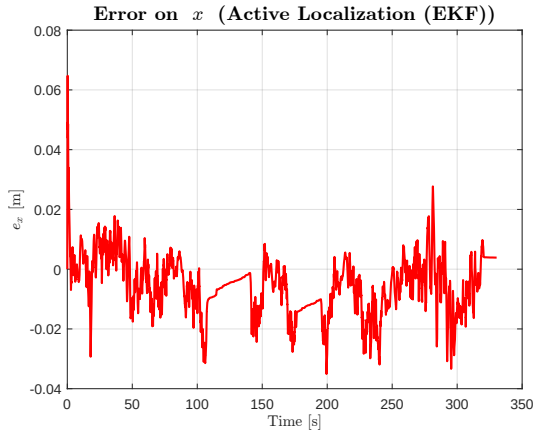


Figure 4.19: x error: EKF case.

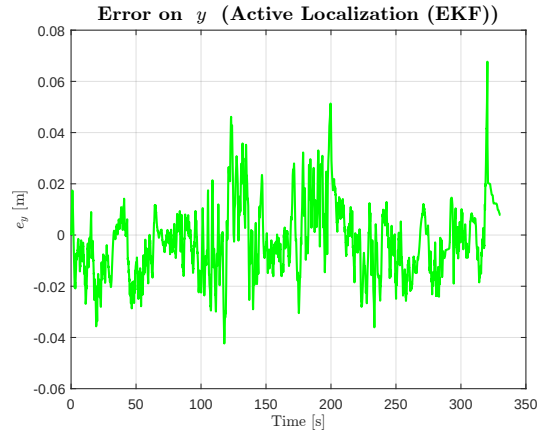


Figure 4.20: y error: EKF case.

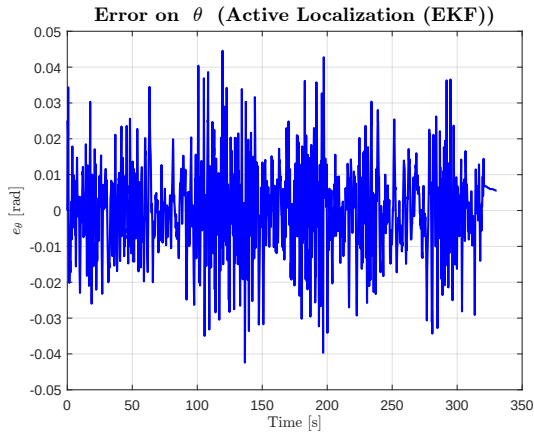


Figure 4.21: θ error: EKF case.

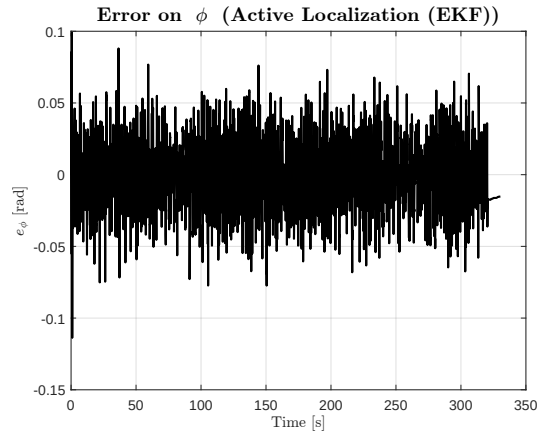


Figure 4.22: ϕ error: EKF case.

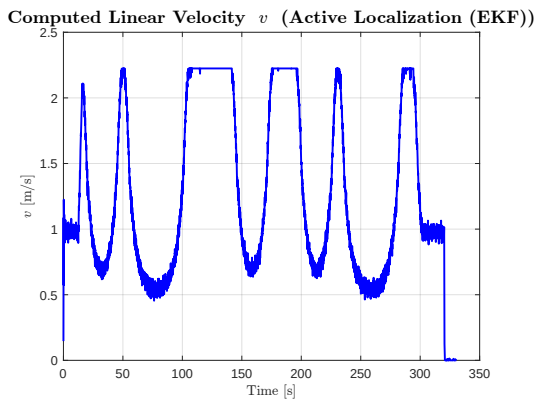


Figure 4.23: $v(t)$ command: EKF Case.

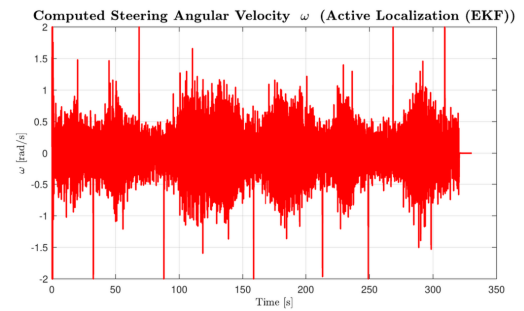


Figure 4.24: $\omega(t)$ command: EKF Case.

Chapter 5

Conclusion

This project successfully developed and evaluated a complete guidance, control, and localization framework for a non-holonomic car-like robot operating in a mapped environment. The systematic integration of advanced trajectory planning, an Input-Output (I/O) Feedback Linearization controller, and various state estimation techniques has provided critical insights into the fundamental interplay between control performance and localization accuracy.

5.1 Evaluation of the I/O Feedback Controller

The implementation of the I/O Feedback Linearization controller demonstrated exceptional efficacy in overcoming the non-holonomic constraints of the car-like kinematic model. By shifting the control objective to a virtual point located ahead of the rear axle, the non-linear, coupled kinematics were successfully transformed into a linear system.

Under nominal conditions with ideal state feedback, the controller achieved near-perfect tracking with minimal transient errors, validating the selection and tuning of the proportional tracking gains ($k_1 = 15, k_2 = 15$). Furthermore, the analytical derivation of the feedforward reference derivatives and the integration of a continuous raised-cosine goal-damping logic proved highly effective in preventing actuator saturation and eliminating high-frequency velocity chattering upon arrival at the destination.

However, the simulation campaigns highlighted that the primary limitation of this control strategy is its extreme vulnerability to state estimation errors. Any degradation or drift in the state estimate directly translates into erroneous control inputs, rapidly destabilizing the tracking path.

5.2 Comparison of Localization Methods

To address the lack of direct state access inherent in physical systems, four distinct feedback configurations were evaluated, revealing critical trade-offs regarding computational complexity, local accuracy, and long-term stability:

- **Ideal Feedback:** This configuration served as the baseline benchmark for the system. While it yielded flawless trajectory tracking and zero error accumulation, it represents an unrealistic scenario for real-world field and service robotics, where states must always be reconstructed from uncertain and noisy sensory data.
- **Forward Euler Odometry:** This method highlighted the severe limitations of low-order, open-loop dead-reckoning. Due to first-order truncation errors, numerical drift accumulated rapidly across the non-linear path. This caused a total breakdown in tracking performance, resulting in an unbounded Euclidean distance error that exceeded 60 meters and drove the robot completely off course.
- **Runge-Kutta 2 (RK2) Odometry:** This approach provided a mathematically superior local integration by exploiting midpoint state approximations, effectively reducing local truncation errors during curved maneuvers. However, because it remains an open-loop strategy, it suffered from the same structural flaw as the Euler method: it could not bound the long-term cumulative drift. Consequently, over an extended execution period, it exhibited an ultimately identical tracking failure, proving that higher-order integration alone cannot solve dead-reckoning drift.
- **Active Localization (EKF):** This architecture demonstrated the absolute necessity of closed-loop state estimation. By fusing the proprioceptive RK2 odometry prediction with absolute exteroceptive range and bearing measurements from a simulated LiDAR sensor, the Extended Kalman Filter successfully bounded the localization drift. The robot accurately followed the reference path and safely reached the designated goal despite stochastic sensor noise. The primary disadvantage of this configuration is the introduction of high-frequency chattering into the control commands (v, ω) , caused by the propagation of measurement noise through the aggressive proportional gains of the I/O controller.

List of Figures

1.1	Car-like kinematic model.	1
2.1	Original map	3
2.2	Binarized map.	3
2.3	Generated Reeds-Shepp graph with the optimal geometric path	6
2.4	Reference trajectory computed	8
2.5	Reference linear velocity $v(t)$	9
2.6	Reference steering angular velocity $\omega(t)$	9
3.1	IO Feedback Linearization Controller: Simulink Scheme.	10
3.2	Trajectory Tracking: Nominal Case	14
3.3	Euclidean tracking error over time: Nominal Case	14
3.4	x error: Nominal case.	15
3.5	y error: Nominal case.	15
3.6	θ error: Nominal case.	15
3.7	ϕ error: Nominal case.	15
3.8	$v(t)$ command: Nominal Case.	15
3.9	$\omega(t)$ command: Nominal Case.	15
4.1	Trajectory Tracking: Euler Odometry case	17
4.2	Euclidean tracking error: Euler Odometry case.	18
4.3	x error: Euler case.	19
4.4	y error: Euler case.	19
4.5	θ error: Euler case.	19
4.6	ϕ error: Euler case.	19
4.7	$v(t)$ command: Euler case.	19
4.8	$\omega(t)$ command: Euler case.	19
4.9	Trajectory Tracking: Runge-Kutta 2 Odometry Case	21
4.10	Euclidean tracking error: Runge-Kutta 2 Odometry case	21
4.11	x error: Runge-Kutta 2 case.	22
4.12	y error: Runge-Kutta 2 case.	22
4.13	θ error: Runge-Kutta 2 case.	22

4.14 ϕ error: Runge-Kutta 2 case.	22
4.15 $v(t)$ command: Runge-Kutta 2 Case.	22
4.16 $\omega(t)$ command: Runge-Kutta 2 Case.	22
4.17 Trajectory Tracking: Active Localization case	25
4.18 Euclidean tracking error: Active Localization case.	25
4.19 x error: EKF case.	26
4.20 y error: EKF case.	26
4.21 θ error: EKF case.	26
4.22 ϕ error: EKF case.	26
4.23 $v(t)$ command: EKF Case.	26
4.24 $\omega(t)$ command: EKF Case.	26
5.1 Simulink Model Overview	30

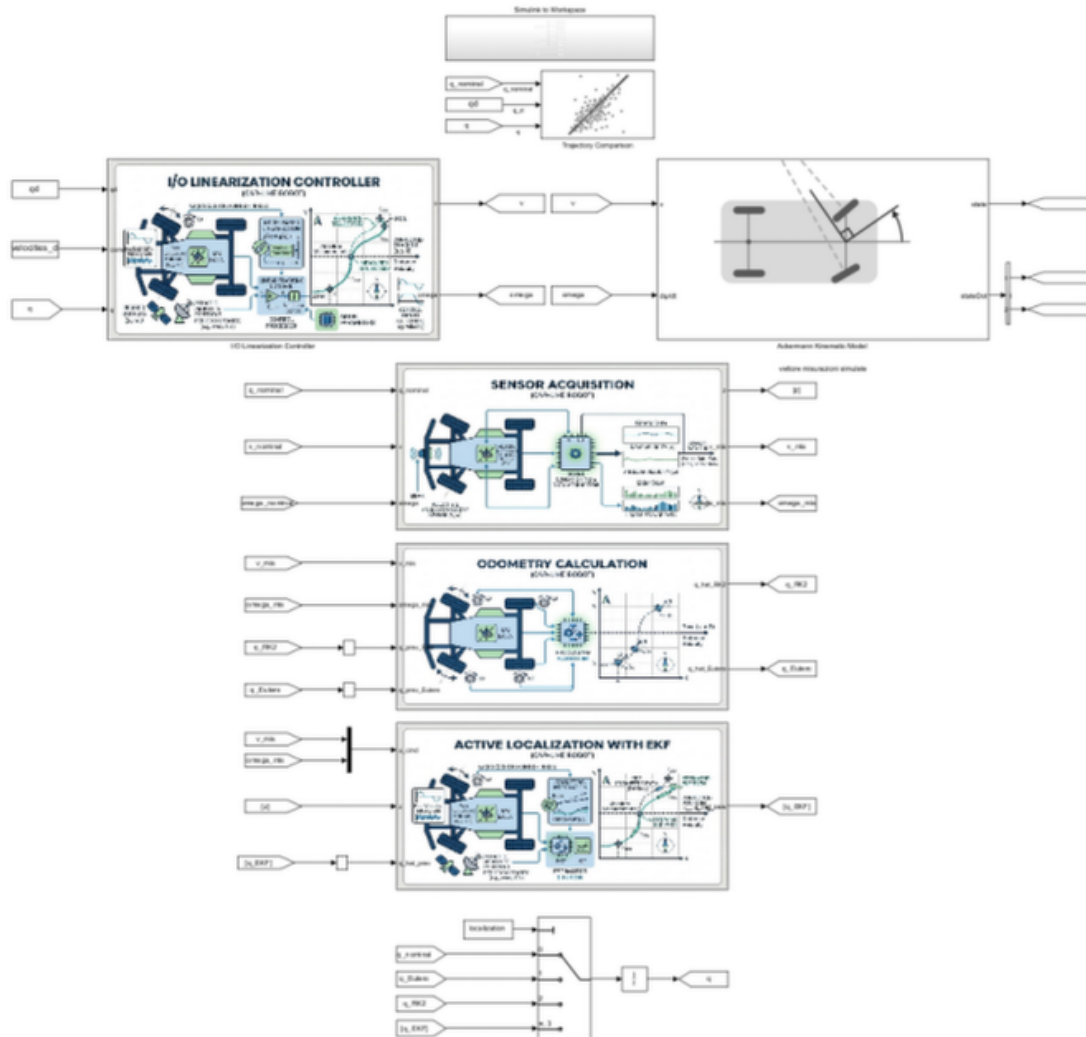


Figure 5.1: Simulink Model Overview